

Assignment 2: Data Stream Processing

Please carefully read the instructions before you begin this assignment.

Course

Data Engineering-II

Introduction

The lab assignment covers the practical part of the discussed concepts of data streaming frameworks. In this lab, we are going to explore one of the popular data streaming frameworks, Apache Pulsar. The lab consists of **three** compulsory tasks, one extra task, and one optional task.

- **Task 1(Compulsory)** provides an understanding of how to set up Apache pulsar on a single Linux virtual Machine.
- **Task 2(Compulsory)** demonstrates the implementation of the messaging system with Apache Pulsar. First, a single consumer and a producer running on the same machine are implemented. The task then introduces students how messaging systems can be distributed between different machines.
- **Task 3(Compulsory)** is a theoretical task that requires an understanding of the concepts of data streaming frameworks i.e., specifically about Apache Pulsar
- **Task 4(Extra)** is a challenging task where students will implement a small use case of Apache pulsar. This task is not compulsory for passing the assignment. **Completing this task will give you one extra point.**
- **Task 5 (Optional – No Credit)** is a challenging task beyond basic usage and explore practical system behavior.

Points

Finishing all compulsory tasks gives you 1 point, and the extra task gives you 1 point. Late submission is allowed for compulsory tasks only. If **minor** fixes are required, you may resubmit both compulsory tasks and extra task until the final deadline.

Submission

For Task 1 and 2, you have to attach the screenshots of your results to the report. Your screenshots should motivate the teaching team that you can successfully set up the infrastructure and run the required tasks. For Task 3, answer all the questions concisely. 2-4 pages should be sufficient for answering all 12 questions, although we will **not** impose this limit strictly. For Task 4, see the details in the task. The submitted report has to be in the PDF format. Also submit all related code files along with your report in the submission box. Do **not** zip or compress the files. Upload them individually to the submission box.

Required/allowed resources

- One VM (medium flavor) for Task 1

- Two VMs (medium flavor) for Task 2

Prerequisites

- Ubuntu 22.04 or higher
- Java Runtime Environment
- Docker latest

Discussions

Problems can be discussed on Slack channel. The forum encourages students to discuss issues related to the lab work. If you have any questions regarding the assignment, please attend the lab sessions on Zoom or ask them on Slack channel.

Note: Assignments can be discussed with your fellow students, but copying solutions of your fellow students is **NOT** allowed, which can result in disciplinary action. The teaching team reserves the right to call for an additional oral examination if we suspect any type of cheating.

External Resources

Students will be required to download source code that will assist in completing the tasks. The source code will be available on a public repository service. Link to relevant repositories will be specified in the assignment question.

Supplementary Material

Information on Apache Pulsar: <https://pulsar.apache.org/docs/en/standalone/>

Relevant literature on data streaming framework: <https://ieeexplore.ieee.org/document/8864052/>

Information on Terminal Multiplexer: <https://linuxize.com/post/getting-started-with-tmux/>

Do not forget to regularly back up your work. Please also follow the policy to use cloud resources. Briefly, the teaching team reserves the right to delete VMs that run for longer than **three days** without prior notice. We will not be responsible for any damage to this.

Task1 (Compulsory): Setting up Apache Pulsar

First, we need to set up an Apache pulsar in our VM. We can either set it up directly in the VM (the bare VM) or set it up in a docker container inside the VM.

A. Start Pulsar in the bare VM

1. Switch to root user and update the package lists

```
$ sudo apt update; sudo apt -y upgrade;
```

2. Download binary release of pulsar by running the following command in your terminal. This step can take a long amount of time.

```
$ wget https://archive.apache.org/dist/pulsar/pulsar-2.9.4/apache-pulsar-2.9.4-bin.tar.gz
```

3. Untar the downloaded tarball

```
$ tar xvfz apache-pulsar-2.9.4-bin.tar.gz
```

4. Start a local cluster with the following command, specifying the standalone mode, where all components of pulsar are running in one single machine.

```
$ apache-pulsar-2.9.4/bin/pulsar standalone
```

5. When pulsar is successfully running, we will see the following status in the terminal:

```
...
{
  "c-standalone-fw-localhost-8080" : {
    "originalNumAssignments" : 0,
    "finalNumAssignments" : 0,
    "instancesAdded" : 0,
    "instancesRemoved" : 0,
    "instancesUpdated" : 0,
    "alive" : true
  }
}
09:49:49.319 [pulsar-web-68-7] INFO  org.eclipse.jetty.server.RequestLog -
127.0.0.1 - - [06/Apr/2025:09:49:49 +0000] "GET
/admin/v2/persistent/public/functions/coordinate/stats?getPreciseBacklog=false
HTTP/1.1" 200 1399 "-" "Pulsar-Java-v2.9.4" 17
...
```

The above log shows the successful setup of the Apache pulsar. To stop Apache pulsar, press Ctrl + C.

B. Setting up pulsar in a container in a VM

1. One of the advantages of using docker containers is to quickly set up a software service. Instead of setting up Pulsar in the bare VM, we can run Apache pulsar in a Docker container with the following docker command:

```
$ docker run -it -p 6650:6650 -p 8080:8080 \  
--mount source=pulsardata,target=/pulsar/data \  
--mount source=pulsarconf,target=/pulsar/conf \  
apachepulsar/pulsar:2.9.4 bin/pulsar standalone
```

Task 1: Note

You can choose to use either method A or B for setting up Pulsar, but we recommend you try both methods. When writing the report, take a screenshot for only one method that you choose.

Task 2 (Compulsory): Producing and Consuming messages

We have a running Pulsar service in VM. Now, we will run both a message-consuming program (i.e. consumer) and a message-producing program (i.e., producer) on the same machine.

Keep the terminal open and Apache Pulsar running. Open two new terminals (one for a consumer and one for a producer) and connect them to the same VM which the Pulsar service is running on.

A. Running Apache Pulsar, consumer and producer on a single machine

1. First step is to install the Python library for Apache Pulsar client. Run the following command in any of the two new terminals:

```
$ pip install pulsar-client==2.9.4
```

2. In one of the new terminals, create a consumer with the following code snippet and keep it running

```
import pulsar

# Create a pulsar client by supplying ip address and port
client = pulsar.Client('pulsar://localhost:6650')
# Subscribe to a topic and subscription
consumer = client.subscribe('DEtopic', subscription_name='DE-sub')
# Display message received from producer
msg = consumer.receive()

try:
    print("Received message : '%s'" % msg.data())

    # Acknowledge for receiving the message
    consumer.acknowledge(msg)

except:
    consumer.negative_acknowledge(msg)
# Destroy pulsar client
client.close()
```

3. In another one of the new terminals, create a producer with the following code snippet, and run it.

```
import pulsar

# Create a pulsar client by supplying ip address and port
client = pulsar.Client('pulsar://localhost:6650')
# Create a producer on the topic that consumer can subscribe to
```

```
producer = client.create_producer('DEtopic')
# Send a message to consumer
producer.send(('Welcome to Data Engineering Course!').encode('utf-8'))
# Destroy pulsar client
client.close()
```

Now, when checking the output in the consumer terminal, you should see the output, with a line in between printing “Welcome to Data Engineering Course!”. Do not forget to take the screenshots.

B. Running Apache pulsar, consumer and producer on multiple machines

In this exercise, we will run a consumer and a producer as shown in the example above on two separate virtual machines. For example, Apache Pulsar and the producer are run on virtual machine A, whereas the consumer is run on virtual machine B. This task requires setting up two virtual machines on the NAISS cloud. To check if the distributed Pulsar setup works i.e., messages are produced and consumed properly, see the output log of the consumer on virtual machine A.

When taking the screenshots, make sure they can clearly show that the consumer and producer run on different machines.

Note: For the communication between the consumer/producer and Apache Pulsar, you need to specify the IP address where Pulsar is running in the client object. Should you use the private IP address (192.128.*.*), or the public/floating IP address (130.*.*.*).

Task 3 (Compulsory): Conceptual questions

1. What are the primary trade-offs when deploying Apache Pulsar in a Docker container versus a native environment?
2. What features does Apache Pulsar support that previous distributed streaming frameworks, such as Kafka, do not?
3. What is the issue with using a batch processing approach on data at scale? How do modern data stream processing systems such as Apache Pulsar can overcome the issue of batch processing?
(Read literature for this question: <https://ieeexplore.ieee.org/document/8864052/>)
4. What is the underlying messaging pattern used by Apache Pulsar? What is the advantage of this messaging pattern?
5. What are different modes of subscription? When is each mode of subscription used?
6. What is the role of ZooKeeper? Is it possible to ensure reliability in streaming frameworks such as Pulsar or Kafka without ZooKeeper?
7. What are the main architectural components of Apache Pulsar?
8. What is the meaning of broker in Pulsar?
9. What is the role of a tenant in Pulsar?
10. What will happen if there is no log compaction in Apache Pulsar?
11. What is the difference between Pulsar topics and partitions?
12. What is the role of acknowledgement in message delivery guarantees?

Task 4 (Extra): Splitting and merging operations with Apache Pulsar

In this task, we are going to demonstrate how Apache Pulsar can be used for splitting and merging operations. For this exercise, please clone the repository by using the following command:

```
$ git clone https://github.com/JesperStromblad/DE2LAB.git
```

The repository consists of “conversion.py”, which demonstrates the code that has a “conversion” operation applied to each word on a string. Here, we assume that the operation can only be applied to a single word instead of the entire string. Based on this premise, you are required to complete the following sub-tasks:

1. Identify an issue with the current implementation in terms of handling big data i.e., words are in the order of millions.
2. Redesign the current implementation by using Apache Pulsar to demonstrate splitting and merging of data i.e., how the same operation on each word (i.e., splitting) and the resultant string (i.e., merging) can be handled by consumers and producers. Provide a diagram to demonstrate how your architecture will look like i.e.,
 - a. How data splitting and merging is handled by consumers and producers
 - b. Label broker, consumer and producer.
3. Provide an implementation (preferably in Python programming language) of your architecture with Apache Pulsar. You can set up your architecture on **a single virtual machine** by creating different sessions for each consumer and producer. Also include screenshots of your results to show that your implementation runs successfully.

Please read the “Submission” section before and after submitting your work to make sure that you submit all required files. In short, you need to submit **your report** and **all related source codes**. If you have any questions regarding grading, please contact the grader that grades your work through their email or Studium message.

Task 5 (Optional – No Credit): Monitoring, Performance, and Reliability

This is an **optional task** intended for exploration and deeper understanding.

It **will not be counted toward extra credit**, and no active support will be provided for this part.

You do not have to write this part into your report.

In this task, you will go beyond basic usage and explore **practical system behavior**.

- Message Throughput Experiment
 - Modify your producer to send at least 1000 messages.
 - Measure
 - Time taken to send messages
 - Time taken by consumers to receive them
- Failure Simulation
 - Stop the consumer temporarily while the producer is sending messages.
 - Restart the consumer and observe the following
 - Are messages lost?
 - Are they re-delivered?
- Acknowledgement Behavior
 - Modify consumers to not acknowledge some messages.
 - Observe what happens to unacknowledged messages.
- Short Analysis (Important)
 - What does this experiment tell you about
 - Reliability
 - Fault Tolerance
 - Message guarantees in Pulsar